

Unidad 6: Excepciones y Programación orientada a Objetos en Python

Concepto de Errores

Diferencias entre Errores y Excepciones en Python

En Python, los términos "errores" y "excepciones" a menudo se usan indistintamente, pero tienen significados específicos y distintos.

Errores

Un error en Python generalmente se refiere a un problema detectado por el intérprete antes de que el programa se ejecute. Estos errores se deben principalmente a problemas en la sintaxis del código. Los errores son graves y detienen la ejecución del programa inmediatamente.

Excepciones

Una excepción, en cambio, es un evento que ocurre durante la ejecución de un programa y que interrumpe el flujo normal del programa. A diferencia de los errores, las excepciones pueden manejarse mediante bloques `try` y `except`, lo que permite que el programa continúe su ejecución incluso después de que se haya producido un problema.

Importancia de Manejar Excepciones

Manejar excepciones es crucial para construir programas robustos y confiables. Sin un manejo adecuado de excepciones, un programa puede detenerse abruptamente, lo que puede resultar en una mala experiencia para el usuario y posibles pérdidas de datos. Al capturar y manejar excepciones, los desarrolladores pueden proporcionar mensajes de error útiles y tomar medidas correctivas, permitiendo que el programa siga funcionando de manera controlada.

```
```python
```

```
try:
```

```
Código que puede causar una excepción
resultado = 10 / 0
except ZeroDivisionError:
 # Manejo de la excepción
 print("No se puede dividir por cero.")

...

```

En este ejemplo, la excepción `ZeroDivisionError` se maneja, lo que evita que el programa se detenga abruptamente.

---

## Tipos de Errores

### Errores Sintácticos

Los errores sintácticos ocurren cuando el código no sigue las reglas de la sintaxis de Python. Son detectados por el intérprete antes de que el programa se ejecute.

#### Ejemplo de Error Sintáctico

```
```python

print("Hola mundo)

...

```

Este código generará un error de sintaxis porque falta la comilla de cierre.

Prevención de Errores Sintácticos

- Usar un editor de código con resaltado de sintaxis.
 - Revisar el código cuidadosamente para asegurarse de que todas las estructuras están correctamente cerradas (paréntesis, comillas, etc.).
 - Ejecutar el código frecuentemente durante el desarrollo para detectar errores tempranamente.
-

Errores de Nombre

Los errores de nombre ocurren cuando el código intenta usar una variable que no ha sido definida. Esto puede deberse a errores tipográficos o a olvidos en la definición de variables.

Ejemplo de Error de Nombre

```
```python  

print(valor)

```
```

Si `valor` no ha sido definido previamente en el código, se producirá un error `NameError`.

Prevención de Errores de Nombre

- Asegurarse de que todas las variables están definidas antes de su uso.
 - Evitar errores tipográficos utilizando nombres de variables consistentes y significativos.
 - Usar herramientas de análisis de código estático que pueden detectar variables no definidas.
-

Errores Semánticos

Los errores semánticos son más difíciles de detectar porque el código se ejecuta sin problemas, pero no produce el resultado esperado. Estos errores ocurren cuando hay un fallo en la lógica del programa.

Ejemplo de Error Semántico

```
```python  

a = 5
b = 10
print("La suma es:", a * b)

```
```

Este código pretende sumar `a` y `b`, pero en su lugar los multiplica, lo que produce un resultado incorrecto.

Prevención de Errores Semánticos

- Escribir pruebas unitarias para verificar que el código produce los resultados esperados.
- Revisar y depurar el código con cuidado para asegurar que la lógica es correcta.
- Utilizar revisiones de código y prácticas de desarrollo en equipo para detectar errores lógicos.

Conclusión

Entender la diferencia entre errores y excepciones, así como conocer los tipos más comunes de errores en Python, es esencial para escribir código robusto y eficiente. El manejo adecuado de excepciones permite a los desarrolladores crear programas que no solo funcionan correctamente, sino que también son capaces de manejar problemas inesperados de manera elegante y efectiva.

Manejo de Excepciones en Python

Introducción a las Excepciones

En Python, las excepciones son eventos que ocurren durante la ejecución de un programa y que interrumpen su flujo normal. Estos eventos suelen ser errores que impiden que el programa continúe, como intentar dividir un número por cero o acceder a un archivo que no existe. Python proporciona un mecanismo para manejar estos errores de manera controlada, permitiendo que el programa no se detenga abruptamente.

¿Qué es un Bloque Try-Except?

El bloque `try-except` es una estructura de control que permite manejar excepciones de manera efectiva. La idea básica es intentar ejecutar un bloque de código y, si ocurre una excepción, capturarla y realizar una acción alternativa. La sintaxis básica es la siguiente:

```
```python
try:
 # Código que puede causar una excepción
except ExceptionType:
 # Código para manejar la excepción
...
```
```

Aquí, el bloque `try` contiene el código que puede generar una excepción, mientras que el bloque `except` contiene el código que se ejecuta si ocurre esa excepción.

Manejo de Excepciones con Try-Except

El uso del bloque `try-except` es fundamental para gestionar los errores de manera controlada. A continuación, veremos cómo utilizarlo con ejemplos que capturan y manejan excepciones comunes.

Ejemplo 1: Manejo de ZeroDivisionError

```
```python

try:
 resultado = 10 / 0
except ZeroDivisionError:
 print("Error: No se puede dividir por cero.")

```
```

En este ejemplo, intentamos dividir 10 entre 0, lo cual genera un `ZeroDivisionError`. Gracias al bloque `except`, capturamos la excepción y mostramos un mensaje de error en lugar de que el programa se detenga abruptamente.

Ejemplo 2: Manejo de FileNotFoundError

```
```python

try:
 with open('archivo_no_existente.txt', 'r') as archivo:
 contenido = archivo.read()
except FileNotFoundError:
 print("Error: El archivo no existe.")

```
```

Aquí intentamos abrir un archivo que no existe, lo que produce un `FileNotFoundError`. La excepción se captura y se muestra un mensaje al usuario.

Uso de Else y Finally en Excepciones

Además de los bloques `try` y `except`, Python ofrece las cláusulas `else` y `finally` para manejar excepciones de manera más completa y robusta.

Bloque Else

El bloque `else` se ejecuta si el bloque `try` no genera ninguna excepción. Es útil para ejecutar código que debería correr solo si no hubo errores.

```
```python

try:
 resultado = 10 / 2
except ZeroDivisionError:
 print("Error: No se puede dividir por cero.")
else:
 print("La operación fue exitosa, el resultado es:", resultado)

```
```

En este ejemplo, dado que no ocurre una excepción, se ejecuta el bloque `else` y se muestra el resultado de la operación.

Bloque Finally

El bloque `finally` se ejecuta siempre, ocurra o no una excepción, lo que es útil para liberar recursos o realizar acciones de limpieza.

```
```python

try:
 archivo = open('mi_archivo.txt', 'w')
 archivo.write("Hola, mundo!")
except IOError:
 print("Error: No se pudo escribir en el archivo.")
finally:
 archivo.close()
 print("El archivo ha sido cerrado.")

```
```

En este ejemplo, el bloque `finally` asegura que el archivo se cierre sin importar si ocurre una excepción o no.

Excepciones Múltiples

En Python, es posible manejar múltiples excepciones en un solo bloque `try-except` utilizando diferentes cláusulas `except` para distintos tipos de errores. Esto permite manejar errores de manera específica y efectiva.

Ejemplo de Manejo de Múltiples Excepciones

```
```python

try:
 valor = int(input("Ingresa un número: "))
 resultado = 10 / valor
except ValueError:
 print("Error: Debes ingresar un número.")
except ZeroDivisionError:
 print("Error: No se puede dividir por cero.")

```
```

En este ejemplo, capturamos dos tipos de excepciones: `ValueError`, que ocurre si el usuario ingresa algo que no es un número, y `ZeroDivisionError`, que ocurre si el usuario ingresa cero.

Ejemplo Detallado de Múltiples Excepciones

```
```python

def procesar_archivo(nombre_archivo):
 try:
 with open(nombre_archivo, 'r') as archivo:
 contenido = archivo.read()
 numero = int(contenido.strip())
 resultado = 100 / numero
 print("El resultado es:", resultado)
 except FileNotFoundError:
 print("Error: El archivo no fue encontrado.")
 except ValueError:
 print("Error: El contenido del archivo no es un número válido.")
 except ZeroDivisionError:
 print("Error: El número en el archivo es cero, lo cual no es permitido.")

```
```

Conclusión

El manejo de excepciones en Python es una habilidad esencial para escribir programas robustos y fiables. Al utilizar bloques `try-except`, los desarrolladores pueden anticipar y controlar los errores, mejorando así la experiencia del usuario y evitando que los programas se interrumpan de manera abrupta. El uso de las cláusulas `else` y `finally` permite una gestión aún más fina de los recursos y los flujos de trabajo, asegurando que el código funcione de manera eficiente incluso en situaciones imprevistas.

Finalmente, la capacidad de manejar múltiples excepciones en un solo bloque proporciona flexibilidad y control para responder de manera específica a una variedad de problemas, lo cual es crucial en el desarrollo de software de calidad.

¿Qué es la POO?

La Programación Orientada a Objetos (POO) es un paradigma de programación que organiza el código en torno a "objetos" y "clases" en lugar de en torno a funciones y procedimientos, como se hace en la programación tradicional (también conocida como programación estructurada).

Definición

En la POO, una clase es un molde que define las características y comportamientos (atributos y métodos) que tendrán los objetos creados a partir de ella. Un objeto, por su parte, es una instancia de una clase, que contiene datos y métodos para manipular esos datos.

Por ejemplo, en una aplicación que maneja información de vehículos, podríamos tener una clase `Vehiculo` con atributos como `marca`, `modelo` y `año`, y métodos como `arrancar()` y `detenerse()`. Cada vehículo específico sería un objeto de la clase `Vehiculo`.

Diferencia con la programación tradicional

La programación tradicional organiza el código en funciones y procedimientos que operan sobre datos. En cambio, la POO agrupa tanto los datos como las funciones que operan sobre esos datos en unidades llamadas objetos.

Beneficios de la POO

1. **Modularidad:** El código se organiza en pequeñas unidades llamadas objetos que son fáciles de gestionar y mantener.
 2. **Reutilización:** Las clases pueden reutilizarse en diferentes programas.
 3. **Escalabilidad:** Facilita la gestión de proyectos grandes y complejos.
 4. **Mantenimiento:** La organización en clases y objetos facilita la localización y corrección de errores.
-

Motivación para Usar POO

La POO se ha convertido en una elección popular para muchos desarrolladores debido a varios factores clave:

Reutilización de código

Una de las mayores ventajas de la POO es la capacidad de reutilizar código. Las clases bien definidas pueden ser usadas en múltiples aplicaciones, ahorrando tiempo y esfuerzo en el desarrollo.

Manejo de aplicaciones complejas

La POO facilita la organización de proyectos grandes y complejos. La capacidad de agrupar datos y comportamientos en objetos hace que el código sea más intuitivo y manejable.

Organización y mantenimiento del código

La estructura modular de la POO permite que diferentes partes de un proyecto sean desarrolladas y probadas de manera independiente. Esto hace que el mantenimiento del código sea más eficiente, ya que los cambios en una parte del código no afectan necesariamente a otras partes.

Diferencias con la Programación Tradicional

Programación Estructurada

En la programación estructurada, el enfoque principal está en las funciones y procedimientos. El flujo del programa se controla a través de llamadas a funciones y la manipulación de variables globales.

Programación Orientada a Objetos

En la POO, el código se estructura alrededor de objetos y clases. Esta estructura permite:

1. **Encapsulación:** Agrupar datos y métodos que operan sobre esos datos en una sola unidad (clase). Esto oculta los detalles internos y muestra solo lo necesario.
2. **Herencia:** Crear nuevas clases basadas en clases existentes, facilitando la reutilización y extensión del código.
3. **Polimorfismo:** Permitir que objetos de diferentes clases sean tratados como objetos de una clase común, haciendo que el código sea más flexible y extensible.
4. **Abstracción de datos:** Permitir a los desarrolladores centrarse en la lógica de alto nivel sin preocuparse por los detalles de implementación.

Ventajas de la POO

- **Encapsulación:** Protege los datos y asegura que se usen de manera adecuada.
 - **Herencia:** Reduce la redundancia y facilita la reutilización de código.
 - **Polimorfismo:** Mejora la flexibilidad del código y su capacidad para manejar diferentes tipos de datos.
 - **Abstracción:** Simplifica el desarrollo al permitir que los desarrolladores trabajen con conceptos de alto nivel.
-

Pensando en POO

Para aplicar el pensamiento orientado a objetos, es útil modelar problemas del mundo real identificando clases, atributos y métodos relevantes.

Identificación de clases

Comienza identificando las entidades principales en tu problema. Por ejemplo, en una aplicación para gestionar una biblioteca, las entidades podrían ser `Libro`, `Usuario` y `Prestamo`.

Definición de atributos

Cada clase debe tener atributos que describan sus características. Por ejemplo, la clase `Libro` podría tener atributos como `titulo`, `autor` y `ISBN`.

Definición de métodos

Los métodos son las acciones que pueden realizar los objetos de una clase. Para la clase `Libro`, los métodos podrían incluir `prestar()`, `devolver()` y `reservar()`.

Ejemplo práctico

Imagina que estás desarrollando una aplicación para una tienda de ropa. Las clases podrían incluir ``Producto``, ``Cliente`` y ``Pedido``. Cada clase tendría sus propios atributos y métodos:

- **Clase Producto:**
 - Atributos: ``nombre``, ``precio``, ``talla``
 - Métodos: ``aplicar_descuento()``, ``actualizar_stock()``
- **Clase Cliente:**
 - Atributos: ``nombre``, ``email``, ``direccion``
 - Métodos: ``registrar()``, ``actualizar_informacion()``
- **Clase Pedido:**
 - Atributos: ``numero_pedido``, ``cliente``, ``lista_productos``
 - Métodos: ``agregar_producto()``, ``remover_producto()``, ``calcular_total()``

Al pensar en POO, organizas el código de manera que cada objeto maneje sus propios datos y comportamientos, lo que resulta en un diseño más claro y mantenible.

Clases

Relaciones entre Clases

En la Programación Orientada a Objetos (POO), las relaciones entre clases son fundamentales para estructurar y organizar el código. Existen varios tipos de relaciones, cada una con sus propias características y usos. Aquí se describen las principales: la agregación, la asociación y la generalización.

Asociación

La asociación es una relación simple entre dos clases que establece cómo interactúan entre sí. Puede ser unidireccional o bidireccional.

Ejemplo: Un ``Profesor`` y un ``Curso`` tienen una relación de asociación porque un profesor puede enseñar varios cursos y un curso puede ser enseñado por varios profesores.

Agregación

La agregación es una forma de asociación que representa una relación "todo/parte" entre el objeto contenedor (todo) y los objetos contenidos (partes). En una agregación, las partes pueden existir independientemente del todo.

Ejemplo: Una ``Clase`` y un ``Estudiante`` tienen una relación de agregación porque una clase está compuesta por varios estudiantes, pero los estudiantes pueden existir independientemente de la clase.

Composición

La composición es una forma más fuerte de agregación. En una relación de composición, las partes no pueden existir independientemente del todo. Si el contenedor se destruye, también se destruyen las partes.

Ejemplo: Un `Coche` y un `Motor` tienen una relación de composición porque un motor es parte integral de un coche. Si el coche se destruye, el motor también se destruye.

Generalización

La generalización es una relación jerárquica entre una clase general (superclase) y una clase más específica (subclase). La subclase hereda atributos y métodos de la superclase.

Ejemplo: Un `Animal` y un `Perro` tienen una relación de generalización porque un perro es una especificación de un animal. `Perro` hereda las características de `Animal`.

Diagramas de Clases

Los diagramas de clases son una herramienta esencial en la POO para visualizar y planificar las relaciones entre clases en un sistema. Estos diagramas muestran las clases del sistema, sus atributos, métodos y las relaciones entre ellas.

Introducción a los Diagramas de Clases

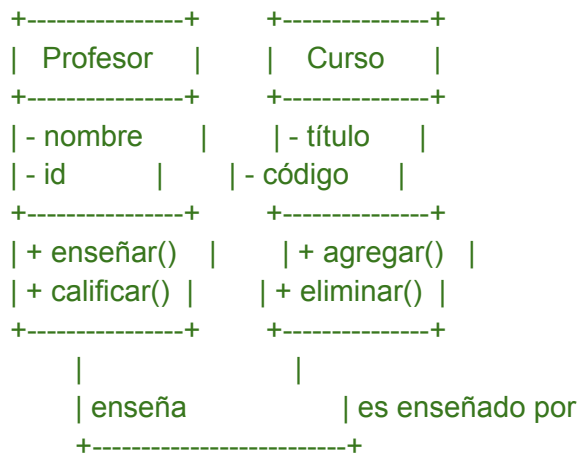
Un diagrama de clases es una representación gráfica que describe la estructura de un sistema mostrando sus clases, atributos, métodos y las relaciones entre los objetos. Los elementos básicos de un diagrama de clases incluyen:

- **Clases:** Representadas como rectángulos con tres compartimentos (nombre de la clase, atributos y métodos).
 - **Relaciones:** Las líneas conectan las clases e indican asociaciones, agregaciones, composiciones o generalizaciones.
-

Ejemplos de Diagramas de Clases

Ejemplo 1: Asociación

...

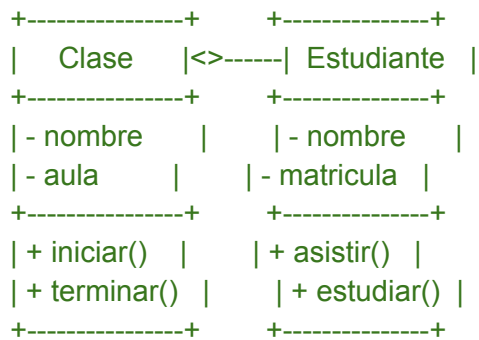


...

En este ejemplo, la relación entre `Profesor` y `Curso` es una asociación bidireccional.

Ejemplo 2: Agregación

...

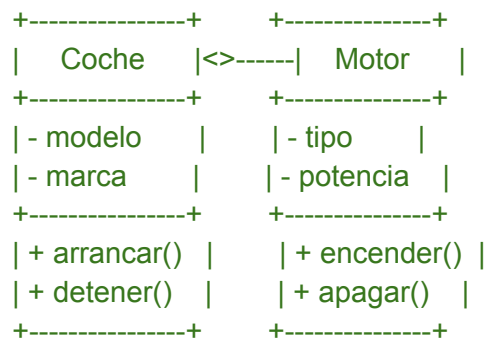


...

Aquí, la relación entre `Clase` y `Estudiante` es una agregación, representada por un diamante vacío.

Ejemplo 3: Composición

...



...

En este caso, la relación entre `Coche` y `Motor` es una composición, indicada por un diamante lleno.

Ejemplo 4: Generalización

...



...

Aquí, `Perro` es una subclase de `Animal` y hereda sus atributos y métodos.

Interpretación de Diagramas de Clases

Al leer un diagrama de clases, es importante prestar atención a:

- **Nombre de las clases:** Identifican las entidades principales del sistema.
- **Atributos y métodos:** Describen las propiedades y comportamientos de las clases.
- **Relaciones:** Muestran cómo las clases interactúan entre sí y qué tipo de relación existe entre ellas (asociación, agregación, composición, generalización).

Los diagramas de clases son una herramienta poderosa para planificar y comprender la estructura de un sistema orientado a objetos, facilitando el diseño y la comunicación entre los desarrolladores.

Material complementario:

Google Colab

- [Repaso Ejercicios](#)
- [Ejemplo Clase](#)

¿Quieres saber más?

- [Errores](#) | Entrenamiento Python
- [Excepciones](#) | Entrenamiento Python
- [Else](#) | Entrenamiento Python
- [Finally](#) | Entrenamiento Python
- [Excepciones integradas](#) | Entrenamiento Python

Tutorial Excepciones

- [Tutorial de Excepciones](#)